

# IDEC MicroSmart / FC6A

## Maintenance Protocol

### Command Structure Reference

Documented commands, HMI-originated requests, program transfer, SD-card operations, force I/O,  
and firmware-transfer structures

Revision 2026-07-28

Prepared from official IDEC protocol documentation, supplied packet captures, and project implementations/tests.

# Contents

|          |                                                                   |          |
|----------|-------------------------------------------------------------------|----------|
| <b>1</b> | <b>Scope and purpose</b>                                          | <b>1</b> |
| <b>2</b> | <b>Base message grammar</b>                                       | <b>1</b> |
| 2.1      | Request framing . . . . .                                         | 1        |
| 2.2      | Reply framing . . . . .                                           | 1        |
| 2.3      | Notation and parsing rules . . . . .                              | 1        |
| <b>3</b> | <b>Confidence and evidence model</b>                              | <b>2</b> |
| <b>4</b> | <b>Confidence A: complete documented or reproduced structures</b> | <b>2</b> |
| 4.1      | Read N bytes: R + operand type . . . . .                          | 2        |
| 4.2      | Write N bytes: W + operand type . . . . .                         | 3        |
| 4.3      | Read one bit: R + lowercase operand type . . . . .                | 3        |
| 4.4      | Write one bit: W + lowercase operand type . . . . .               | 4        |
| 4.5      | Read PLC operating status: RS . . . . .                           | 4        |
| 4.6      | Read scan time: RK . . . . .                                      | 5        |
| 4.7      | Read PLC system program version: RN . . . . .                     | 5        |
| 4.8      | Read error code: RE . . . . .                                     | 6        |
| 4.9      | Clear operand area or system reset: C + type . . . . .            | 6        |
| 4.10     | Read calendar/clock values: RW . . . . .                          | 7        |
| 4.11     | Write calendar/clock values: WW . . . . .                         | 7        |
| 4.12     | Read timer information: R_ . . . . .                              | 8        |
| 4.13     | Read counter information: R' . . . . .                            | 8        |
| 4.14     | Read all timer preset-change flags: Ra . . . . .                  | 9        |
| 4.15     | Read all counter preset-change flags: Rb . . . . .                | 9        |
| 4.16     | Read one timer timeout flag: Rd . . . . .                         | 9        |
| 4.17     | Read one counter countout flag: Re . . . . .                      | 10       |
| 4.18     | Commit changed timer/counter presets: WQ . . . . .                | 10       |
| 4.19     | Legacy user-program protection control: WV . . . . .              | 11       |
| 4.20     | Legacy write user program in ASCII format: WP . . . . .           | 11       |
| 4.21     | Legacy write user program in binary format: Wp . . . . .          | 12       |
| 4.22     | Legacy read user program in ASCII format: RP . . . . .            | 12       |
| 4.23     | Legacy read user program in binary format: Rp . . . . .           | 13       |
| 4.24     | Open/list an SD-card directory: FR . . . . .                      | 13       |
| 4.25     | Read SD-card directory entries: FR20 / FR21 . . . . .             | 14       |
| 4.26     | Resolve/check an SD-card path: FR005 . . . . .                    | 14       |
| 4.27     | Delete an SD-card file or directory: FC005 . . . . .              | 15       |
| 4.28     | Open an SD-card file for reading: Fu . . . . .                    | 15       |
| 4.29     | Read SD-card file blocks: FU . . . . .                            | 15       |
| 4.30     | Create/open an SD-card file for writing: FD . . . . .             | 16       |
| 4.31     | Write SD-card file blocks: FD . . . . .                           | 16       |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 4.32 Create an SD-card directory: FM . . . . .                    | 17        |
| <b>5 Confidence B: strong capture-derived structures</b>          | <b>17</b> |
| 5.1 Read extended register area: RA . . . . .                     | 17        |
| 5.2 Write extended register area: WA . . . . .                    | 18        |
| 5.3 Enable or suspend Force I/O mode: WO . . . . .                | 18        |
| 5.4 Force one physical output: W] . . . . .                       | 19        |
| 5.5 FC6A program segment declaration: WPn . . . . .               | 19        |
| 5.6 FC6A user-program data blocks . . . . .                       | 20        |
| 5.7 FC6A secondary program segment declaration: W;n . . . . .     | 20        |
| 5.8 FC6A transfer status/finalization poll: Ru . . . . .          | 21        |
| 5.9 Firmware binary stream declaration: STX/WPB . . . . .         | 21        |
| 5.10 Firmware binary blocks: STX/WPB . . . . .                    | 22        |
| <b>6 Confidence C: exact structures with incomplete semantics</b> | <b>22</b> |
| 6.1 HMI multi-bit read: Rl . . . . .                              | 22        |
| 6.2 FC6A enhanced-security response: continuation-2 WV . . . . .  | 23        |
| 6.3 FC6A region/session control: Ri . . . . .                     | 23        |
| 6.4 Extended SD status poll: RA1F5F0220 . . . . .                 | 24        |
| <b>7 Confidence D: exact observed frames only</b>                 | <b>24</b> |
| 7.1 Firmware transfer mode/control: WS77FF05 / WS77FF01 . . . . . | 24        |
| <b>8 Observed HMI request spellings</b>                           | <b>25</b> |
| <b>9 Error and result codes</b>                                   | <b>25</b> |
| 9.1 Documented ACK-NG codes . . . . .                             | 25        |
| 9.2 Documented NAK communication errors . . . . .                 | 25        |
| <b>10 Coverage matrix</b>                                         | <b>26</b> |
| <b>11 Known coverage boundaries</b>                               | <b>27</b> |
| <b>A Evidence sources</b>                                         | <b>27</b> |
| <b>B Packet-capture facts used in this revision</b>               | <b>28</b> |
| <b>C Compact grammar for automated readers</b>                    | <b>28</b> |

# 1 Scope and purpose

This document is a consolidated inventory of PLC-side IEC MicroSmart and FC6A Maintenance Protocol command structures covered by the project. It includes commands sent directly by WindLDR and project tools, plus native PLC requests generated by IEC HMIs; the separate HMI maintenance/update protocol on TCP port 2537 is outside this document.

Entries are ordered by confidence, from complete documented or reproduced structures to exact frames whose field semantics remain incomplete. The purpose is coverage tracking: unresolved fields are identified explicitly, but no future-work priorities or speculative command meanings are proposed.

**Operational caution.** Write, clear, program-transfer, force-I/O, and firmware commands can stop a PLC, alter process outputs, erase data, or leave equipment unbootable. Use an isolated test controller and preserve a known-good program and firmware recovery path.

## 2 Base message grammar

### 2.1 Request framing

The normal request frame is:

ENQ DEV CONT CMD [TYPE] [PAYLOAD] BCC2 CR

|                   |                                                                                                                                                                                                                                                                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ENQ</b>        | One byte, 05h. The captured firmware WPB stream is the known exception and begins with STX, 02h.                                                                                                                                                                                                             |
| <b>DEV</b>        | Two ASCII hexadecimal characters. FF is commonly used for a direct/1:1 request; replies identify the responding device as 00 through 1F.                                                                                                                                                                     |
| <b>CONT</b>       | Usually ASCII 0 for final/discontinued or 1 for continued. ASCII 2 is observed in the FC6A enhanced-security WV request and is not treated as a generic continuation rule.                                                                                                                                   |
| <b>CMD / TYPE</b> | In the documented core grammar, CMD is normally R, W, or C and TYPE selects the operand or operation. Extensions such as F-based SD commands and multi-character program-transfer suffixes must be parsed by their complete known structure.                                                                 |
| <b>BCC2</b>       | Two uppercase ASCII hexadecimal characters representing an XOR checksum. Current FC6A captures and project parsers calculate across the leading control byte through the last payload byte; some links may require request BCC calculation without ENQ, so transport code should validate and lock the mode. |
| <b>CR</b>         | One byte, 0Dh.                                                                                                                                                                                                                                                                                               |

A complete captured status request is 05 46 46 30 52 53 33 34 0D, or <ENQ>FF0RS34<CR>. FC6A Ethernet maintenance communication normally carries this same frame directly over TCP port 2101, with no additional application header.

### 2.2 Reply framing

Success: ACK DEV 0 DATA BCC2 CR  
Continued: ACK DEV 1 DATA BCC2 CR  
PLC error: ACK DEV 2 NG2 BCC2 CR  
Link error: NAK DEV 0 ERR2 BCC2 CR

ACK is 06h and NAK is 15h. A PLC-side error therefore still begins with ACK; result character 2 and the following two-character NG code distinguish it from a successful reply.

### 2.3 Notation and parsing rules

- Field widths are suffixes in this document: ADDR4 is four characters, TOTAL8 is eight, and LEN2 is two.

- Unless stated otherwise, numeric fields are uppercase ASCII hexadecimal. The documented standard operand field is four ASCII digits and is decimal for D/T/C/W; X/Y and legacy bit areas use the vendor addressing convention, whose least-significant digit may be octal.
- **DATA\_HEX** means two printable hexadecimal characters per binary byte. **BINARY\_DATA** means unencoded bytes and requires a binary-safe receiver.
- Case and punctuation are protocol data. **RM**, **Rm**, **R\_**, **R'**, **W;n**, and **W]** are distinct structures.

### 3 Confidence and evidence model

| Grade | Meaning            | Use in this document                                                                                                                        |
|-------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| A     | Complete structure | Officially documented and/or reproduced in project code with complete field boundaries and operation meaning.                               |
| B     | Strong structure   | Exact capture or implementation supports the structure and principal role, but model coverage or some response semantics remain incomplete. |
| C     | Partial semantics  | Exact request shape is known, but one or more fields, address mappings, or algorithms are unresolved.                                       |
| D     | Exact frame only   | The bytes are known; the operation name or parameter meanings are intentionally left unassigned.                                            |

Evidence identifiers are listed in Appendix A. “Project status” records what has actually been documented, captured, parsed, or implemented; it is not a development priority list.

## 4 Confidence A: complete documented or reproduced structures

### 4.1 Read N bytes: R + operand type

|                       |                                                                      |
|-----------------------|----------------------------------------------------------------------|
| <b>Command ID</b>     | CORE-RNBYTE                                                          |
| <b>Confidence</b>     | A - documented; repeatedly implemented and observed                  |
| <b>Evidence</b>       | S1, S5, S6                                                           |
| <b>Project status</b> | Implemented in MiSmSerial/MiSmTCP; HMI-originated RD and RM observed |

#### Wire structure.

```
ENQ DEV 0 R TYPE ADDR4 LEN2 BCC2 CR
```

Examples (body only):

```
DEVORD391402      read 2 bytes at D3914
DEVORM108002      read 2 bytes starting at M1080
```

**Description.** Reads a consecutive byte range from the selected operand area. The apparent HMI commands RD and RM are the base R command followed by data type D or M, not separate top-level opcodes.

#### Fields and interpretation.

|              |                                                                                                                                    |
|--------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>TYPE</b>  | Official types: X, Y, M, R, T, t, C, c, D, W. Project extensions A and l are cataloged separately below.                           |
| <b>ADDR4</b> | Four ASCII operand digits in the documented grammar; X/Y/M/R addressing follows the device conventions, while D/T/C/W are decimal. |
| <b>LEN2</b>  | Byte count as two ASCII hexadecimal digits; official maximum is C8 hex (200 bytes).                                                |

**Reply.** ACK DEV 0 DATA\_HEX BCC2 CR. DATA\_HEX contains two ASCII hexadecimal characters for each returned byte.

**Notes.** For M, X, Y, and R byte-block reads, the reply is a packed bit field. The least significant bit of each returned byte corresponds to the first operand in that 8-bit group.

## 4.2 Write N bytes: W + operand type

|                       |                                                              |
|-----------------------|--------------------------------------------------------------|
| <b>Command ID</b>     | CORE-WNBYTE                                                  |
| <b>Confidence</b>     | A - documented; implemented and observed                     |
| <b>Evidence</b>       | S1, S5, S6                                                   |
| <b>Project status</b> | Implemented; HMI-originated WD, WM, and calendar WW observed |

### Wire structure.

```
ENQ DEV 0 W TYPE ADDR4 LEN2 DATA_HEX BCC2 CR
```

Examples (body only):

```
DEV0WD0100021234
```

```
DEV0WM012002A024
```

```
DEV0WW00000E<14 bytes as ASCII hex>
```

**Description.** Writes a consecutive byte range into an operand area. WD writes word data registers, WM writes a packed byte block of relay bits, and WW writes calendar/clock operands.

### Fields and interpretation.

|                 |                                                      |
|-----------------|------------------------------------------------------|
| <b>TYPE</b>     | Official types: X, Y, M, R, T, t, C, c, D, W.        |
| <b>ADDR4</b>    | First operand address.                               |
| <b>LEN2</b>     | Number of binary bytes represented by DATA_HEX.      |
| <b>DATA_HEX</b> | Exactly 2 x LEN bytes of uppercase hexadecimal text. |

**Reply.** ACK DEV 0 BCC2 CR on success; ACK DEV 2 NG BCC2 CR on a PLC-side range, preset-value, or calendar error.

**Notes.** For bit areas, data is written in 8-bit groups. For 16-bit word areas, each word occupies four hexadecimal characters.

## 4.3 Read one bit: R + lowercase operand type

|                       |                                                       |
|-----------------------|-------------------------------------------------------|
| <b>Command ID</b>     | CORE-R1BIT                                            |
| <b>Confidence</b>     | A - documented; implemented and observed              |
| <b>Evidence</b>       | S1, S3, S4, S5                                        |
| <b>Project status</b> | Implemented; Rm observed in program-transfer captures |

### Wire structure.

```
ENQ DEV 0 R type ADDR4 BCC2 CR
```

Observed body: FF0Rm0000

**Description.** Reads one input, output, internal relay, or shift-register bit. The lowercase type character is significant: Rm is a single-bit read, while RM is a byte-block read.

**Fields and interpretation.**

**type** x, y, m, or r.  
**ADDR4** Single operand number.

**Reply.** ACK DEV 0 VALUE BCC2 CR, where VALUE is ASCII 0 or 1.

**Notes.** The captured Rm0000 and Rm8000 requests are structurally identical to the documented single-bit command.

**4.4 Write one bit: W + lowercase operand type**

**Command ID** CORE-W1BIT  
**Confidence** A - documented; implemented and observed  
**Evidence** S1, S3, S4, S5, S6  
**Project status** Implemented; used for HMI buttons and PLC stop/run control

**Wire structure.**

ENQ DEV 0 W type ADDR4 VALUE BCC2 CR

Observed bodies:

FF0Wm80000 reset M8000; stop PLC  
 FF0Wm80001 set M8000; run PLC

**Description.** Sets or resets one bit operand. Wm is widely observed from the HMI and is also used to stop or run the PLC through special relay M8000.

**Fields and interpretation.**

**type** x, y, m, or r.  
**VALUE** ASCII 0 for OFF or 1 for ON.

**Reply.** ACK DEV 0 BCC2 CR on success.

**Notes.** Do not confuse Wm with WM: Wm carries one address and one Boolean value; WM carries a byte length and packed hexadecimal data.

**4.5 Read PLC operating status: RS**

**Command ID** SYS-RS  
**Confidence** A - documented; repeatedly captured and tested  
**Evidence** S1, S3, S4, S5  
**Project status** Implemented and used as a session preflight

**Wire structure.**

ENQ DEV 0 R S BCC2 CR

Complete captured frame:  
 05 FFORS 34 0D

**Description.** Returns PLC run/stop state together with program-protection state and other program metadata. It is commonly the first request in WindLDR sessions and appears repeatedly during program and firmware transfers.

### Fields and interpretation.

**Request payload** None after the S data type.

**Documented reply data**

Operating state, timer/counter preset-change flag, protection mode, CPU type, CRC, and block sum-check values.

**Reply.** ACK DEV 0 STATUS\_DATA BCC2 CR; no documented NG reply.

**Notes.** FC6A reply length and individual metadata fields should be interpreted from the actual model response rather than assuming the older CPU-type table is exhaustive.

## 4.6 Read scan time: RK

**Command ID** SYS-RK

**Confidence** A - documented; tested in the project

**Evidence** S1, S5

**Project status** Implemented/verified; observed reply included current and maximum values

### Wire structure.

ENQ DEV 0 R K BCC2 CR

**Description.** Reads current and maximum PLC scan time. The documented reply contains two 16-bit hexadecimal values measured in milliseconds.

### Fields and interpretation.

**Request payload** None after K.

**Reply data** CURRENT4 MAXIMUM4.

**Reply.** ACK DEV 0 CURRENT4 MAXIMUM4 BCC2 CR; no documented NG reply.

## 4.7 Read PLC system program version: RN

**Command ID** SYS-RN

**Confidence** A - documented; repeatedly captured and tested

**Evidence** S1, S3, S4, S5

**Project status** Implemented and used before protection/program operations

### Wire structure.

ENQ DEV 0 R N BCC2 CR

Captured body: FF0RN

**Description.** Reads the PLC system-program or firmware version field. The project has repeatedly observed RN immediately after RS in WindLDR program-transfer sessions.

### Fields and interpretation.

**Reply data** VERSION4, represented as four hexadecimal characters.



**Reply.** ACK DEV 0 VERSION4 BCC2 CR; no documented NG reply.

#### 4.8 Read error code: RE

|                       |                                                        |
|-----------------------|--------------------------------------------------------|
| <b>Command ID</b>     | SYS-RE                                                 |
| <b>Confidence</b>     | A - documented                                         |
| <b>Evidence</b>       | S1                                                     |
| <b>Project status</b> | Structure covered; not a focus of current capture work |

**Wire structure.**

ENQ DEV 0 R E BCC2 CR

**Description.** Reads the PLC error code. The returned error value is represented as hexadecimal text in the normal ACK data field.

**Fields and interpretation.**

**Request payload** None after E.

**Reply.** ACK DEV 0 ERROR\_DATA BCC2 CR.

**Notes.** The exact width and interpretation are model-dependent; the legacy manual provides the command structure rather than a complete FC6A error dictionary.

#### 4.9 Clear operand area or system reset: C + type

|                       |                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------|
| <b>Command ID</b>     | SYS-CLEAR                                                                                 |
| <b>Confidence</b>     | A - documented                                                                            |
| <b>Evidence</b>       | S1                                                                                        |
| <b>Project status</b> | Structure covered; destructive forms intentionally separated from routine read/write APIs |

**Wire structure.**

ENQ DEV 0 C TYPE BCC2 CR

**Important forms:**

|        |                                                   |
|--------|---------------------------------------------------|
| DEVOCE | clear error code                                  |
| DEVOCZ | system reset / clear all documented operand areas |
| DEVOCI | execute link formatting sequence                  |

**Description.** Clears an entire selected operand area rather than a single address. TYPE Z performs the documented system reset that clears all listed operand areas, while I executes the data-link formatting/initialization sequence.

**Fields and interpretation.**

|                   |                                                        |
|-------------------|--------------------------------------------------------|
| <b>TYPE</b>       | X, Y, M, R, T, t, C, c, D, E, Z, or I.                 |
| <b>No address</b> | This command operates on the selected area as a whole. |

**Reply.** ACK DEV 0 BCC2 CR, or ACK DEV 2 09 BCC2 CR for a data-clear error.

**Notes.** CZ is the documented all-operands reset. It should not be treated as synonymous with the capture-derived Ri command.

#### 4.10 Read calendar/clock values: RW

|                       |                                                  |
|-----------------------|--------------------------------------------------|
| <b>Command ID</b>     | CLOCK-RW                                         |
| <b>Confidence</b>     | A - documented                                   |
| <b>Evidence</b>       | S1                                               |
| <b>Project status</b> | Structure covered; corresponding WW was captured |

##### Wire structure.

```
ENQ DEV 0 R W ADDR4 LEN2 BCC2 CR
```

**Description.** Reads calendar/clock operands using the normal N-byte read grammar with data type W. Operand numbers 0000 through 0006 represent year, month, day, day of week, hour, minute, and second.

##### Fields and interpretation.

|              |                                                  |
|--------------|--------------------------------------------------|
| <b>ADDR4</b> | First calendar/clock field, 0000 through 0006.   |
| <b>LEN2</b>  | Byte count; values are returned as 16-bit words. |

**Reply.** ACK DEV 0 DATA\_HEX BCC2 CR.

#### 4.11 Write calendar/clock values: WW

|                       |                                  |
|-----------------------|----------------------------------|
| <b>Command ID</b>     | CLOCK-WW                         |
| <b>Confidence</b>     | A - documented; captured         |
| <b>Evidence</b>       | S1, S3, S4                       |
| <b>Project status</b> | Observed during WindLDR sessions |

##### Wire structure.

```
ENQ DEV 0 W W ADDR4 LEN2 DATA_HEX BCC2 CR
```

Captured bodies began:  
FF0WW00000E...

**Description.** Writes calendar/clock values using N-byte write data type W. WindLDR sent a 0E-byte write beginning at operand 0000 during both captured program-transfer sessions.

##### Fields and interpretation.

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| <b>DATA_HEX</b>   | Word values for the selected clock fields.                    |
| <b>Validation</b> | The PLC can return NG 08 for an invalid calendar/clock value. |

**Reply.** ACK DEV 0 BCC2 CR, or ACK DEV 2 08 BCC2 CR.

## 4.12 Read timer information: R\_\_

|                       |                                   |
|-----------------------|-----------------------------------|
| <b>Command ID</b>     | TC-RINFO                          |
| <b>Confidence</b>     | A - documented; observed from HMI |
| <b>Evidence</b>       | S1, S6                            |
| <b>Project status</b> | Observed for HMI timer reads      |

### Wire structure.

```
ENQ DEV 0 R _ FIRST4 COUNT2 BCC2 CR
```

**Description.** Reads current value, preset value, and a status byte for consecutive timers. Up to 48 timer records can be returned in one documented request.

### Fields and interpretation.

|               |                                                 |
|---------------|-------------------------------------------------|
| <b>FIRST4</b> | First timer number.                             |
| <b>COUNT2</b> | Number of timers, 01 through 30 hex.            |
| <b>Record</b> | CURRENT4 PRESET4 STATUS2, repeated COUNT times. |

**Reply.** ACK DEV 0 RECORDS BCC2 CR; NG 06 indicates range error and NG 10 indicates invalid data.

## 4.13 Read counter information: R`

|                       |                                                    |
|-----------------------|----------------------------------------------------|
| <b>Command ID</b>     | TC-RCINFO                                          |
| <b>Confidence</b>     | A - documented                                     |
| <b>Evidence</b>       | S1                                                 |
| <b>Project status</b> | Structure covered; no current HMI example retained |

### Wire structure.

```
ENQ DEV 0 R ` FIRST4 COUNT2 BCC2 CR
```

**Description.** Reads current value, preset value, and status for consecutive counters. The backtick character is the documented data type and must be preserved exactly.

### Fields and interpretation.

|               |                                                 |
|---------------|-------------------------------------------------|
| <b>FIRST4</b> | First counter number.                           |
| <b>COUNT2</b> | Number of counters, 01 through 30 hex.          |
| <b>Record</b> | CURRENT4 PRESET4 STATUS2, repeated COUNT times. |

**Reply.** ACK DEV 0 RECORDS BCC2 CR; NG 06 indicates range error and NG 10 indicates invalid data.

#### 4.14 Read all timer preset-change flags: Ra

|                       |                   |
|-----------------------|-------------------|
| <b>Command ID</b>     | TC-RA             |
| <b>Confidence</b>     | A - documented    |
| <b>Evidence</b>       | S1                |
| <b>Project status</b> | Structure covered |

##### Wire structure.

ENQ DEV 0 R a BCC2 CR

**Description.** Returns the preset-value change flags for all documented timers as one packed bitmap. The legacy reply is 26 ASCII hexadecimal characters representing 13 status bytes.

##### Fields and interpretation.

**Request payload** None after a.

**Bit meaning** 1 means the preset value changed; 0 means it did not change.

**Reply.** ACK DEV 0 FLAGS26 BCC2 CR; no documented NG reply.

#### 4.15 Read all counter preset-change flags: Rb

|                       |                   |
|-----------------------|-------------------|
| <b>Command ID</b>     | TC-RB             |
| <b>Confidence</b>     | A - documented    |
| <b>Evidence</b>       | S1                |
| <b>Project status</b> | Structure covered |

##### Wire structure.

ENQ DEV 0 R b BCC2 CR

**Description.** Returns preset-value change flags for all documented counters as one packed bitmap. Its layout mirrors Ra, but each bit refers to a counter.

##### Fields and interpretation.

**Request payload** None after b.

**Reply data** FLAGS26 in the legacy protocol.

**Reply.** ACK DEV 0 FLAGS26 BCC2 CR; no documented NG reply.

#### 4.16 Read one timer timeout flag: Rd

|                       |                   |
|-----------------------|-------------------|
| <b>Command ID</b>     | TC-RD             |
| <b>Confidence</b>     | A - documented    |
| <b>Evidence</b>       | S1                |
| <b>Project status</b> | Structure covered |

##### Wire structure.

ENQ DEV 0 R d TIMER4 BCC2 CR

**Description.** Reads the timeout state of one timer without reading its numeric values. The reply is a single ASCII Boolean.

**Fields and interpretation.**

**TIMER4**            Timer number.  
**VALUE**            0 means not timed out; 1 means timed out.

**Reply.** ACK DEV 0 VALUE BCC2 CR; NG 06 indicates an invalid timer number.

#### 4.17 Read one counter countout flag: Re

**Command ID**        TC-RE  
**Confidence**        A - documented  
**Evidence**           S1  
**Project status**     Structure covered

**Wire structure.**

ENQ DEV 0 R e COUNTER4 BCC2 CR

**Description.** Reads the countout state of one counter. The reply is 0 when not counted out and 1 when counted out.

**Fields and interpretation.**

**COUNTER4**        Counter number.

**Reply.** ACK DEV 0 VALUE BCC2 CR; NG 06 indicates an invalid counter number.

#### 4.18 Commit changed timer/counter presets: WQ

**Command ID**        TC-WQ  
**Confidence**        A - documented  
**Evidence**           S1  
**Project status**     Structure covered

**Wire structure.**

ENQ DEV 0 W Q 0000 02 00 BCC2 CR

**Description.** Copies changed timer and counter preset values from RAM to EEPROM. All operand, length, and data fields in the documented request are fixed.

**Fields and interpretation.**

**Operand**            0000 fixed.  
**Length**            02 fixed.  
**Data**                00 fixed.

**Reply.** ACK DEV 0 BCC2 CR; the legacy manual documents no NG reply.

## 4.19 Legacy user-program protection control: WV

|                       |                                                                 |
|-----------------------|-----------------------------------------------------------------|
| <b>Command ID</b>     | PROTECT-WV-LEGACY                                               |
| <b>Confidence</b>     | A - documented legacy structure                                 |
| <b>Evidence</b>       | S1                                                              |
| <b>Project status</b> | Documented; distinct from the FC6A enhanced-security WV variant |

### Wire structure.

```
ENQ DEV 0 W V PROTECT_CODE8 OPTION BCC2 CR
```

**Description.** Temporarily disables or restores a protection mode already configured in the PLC. The legacy command sends an eight-byte protect code directly, followed by option 0 to disable or 1 to enable.

### Fields and interpretation.

#### PROTECT\_CODE8

Eight raw ASCII bytes; short codes are NUL padded in the legacy specification.

**OPTION** ASCII 0 disables temporarily; ASCII 1 restores the configured protection.

**Reply.** ACK DEV 0 BCC2 CR, or ACK DEV 2 05 BCC2 CR for an incorrect protect code.

**Notes.** Do not send a plaintext legacy password using the enhanced-security FC6A frame described later; the captured FC6A form has a different continuation value and payload length.

## 4.20 Legacy write user program in ASCII format: WP

|                       |                                                                |
|-----------------------|----------------------------------------------------------------|
| <b>Command ID</b>     | PROG-WP-ASCII                                                  |
| <b>Confidence</b>     | A - documented legacy structure                                |
| <b>Evidence</b>       | S1                                                             |
| <b>Project status</b> | Documented; not the FC6A WPn transfer used in current captures |

### Wire structure.

Phase 1:

```
ENQ DEV 1 W P CPU_TYPE CAPACITY8 BCC2 CR
```

Phase 2:

```
ENQ DEV 0 ASCII_PROGRAM_DATA BCC2 CR
```

**Description.** Declares and then writes an ASCII-code user-program image in two request phases. The second phase has no W command byte; it consists of the final continuation marker followed directly by program data.

### Fields and interpretation.

**CPU\_TYPE** One legacy CPU-model code.

**CAPACITY8** Program capacity as eight hexadecimal characters.

#### ASCII\_PROGRAM\_DATA

Up to the legacy documented maximum of 64,336 bytes.

**Reply.** Each phase receives ACK DEV 0 BCC2 CR; phase 1 can return program-size, protection, RUN, CRC, or CPU-type NG codes.

## 4.21 Legacy write user program in binary format: Wp

|                       |                                                                |
|-----------------------|----------------------------------------------------------------|
| <b>Command ID</b>     | PROG-WP-BINARY                                                 |
| <b>Confidence</b>     | A - documented legacy structure                                |
| <b>Evidence</b>       | S1                                                             |
| <b>Project status</b> | Documented; not the FC6A WPn transfer used in current captures |

### Wire structure.

Phase 1:

```
ENQ DEV 1 W p CPU_TYPE CAPACITY8 BCC2 CR
```

Phase 2:

```
ENQ DEV 0 BINARY_PROGRAM_DATA BCC2 CR
```

**Description.** Declares and writes a raw binary user-program image in two phases. It differs from WP only by lowercase p and by allowing binary bytes in the second phase.

### Fields and interpretation.

#### BINARY\_PROGRAM\_DATA

Raw bytes, not ASCII hexadecimal text.

#### Legacy maximum

32,168 bytes in the cited manual.

**Reply.** Each phase receives ACK DEV 0 BCC2 CR; phase 1 uses the same program-transfer NG family as WP.

## 4.22 Legacy read user program in ASCII format: RP

|                       |                                                                  |
|-----------------------|------------------------------------------------------------------|
| <b>Command ID</b>     | PROG-RP-ASCII                                                    |
| <b>Confidence</b>     | A - documented legacy structure                                  |
| <b>Evidence</b>       | S1                                                               |
| <b>Project status</b> | Documented; current FC6A upload data phase not capture-validated |

### Wire structure.

Request phase 1:

```
ENQ DEV 1 R P CPU_TYPE CAPACITY8 BCC2 CR
```

Request phase 2:

```
ENQ DEV 0 R BCC2 CR
```

**Description.** Requests an ASCII user-program upload in two transactions. The first reply declares CPU type and actual capacity with result 1, and the second reply returns the program data with result 0.

### Fields and interpretation.

**CAPACITY8** Caller-provided buffer capacity; must be large enough.

**Phase 2 request** The single R byte follows continuation 0; no data type is present.

**Reply.** Reply 1: ACK DEV 1 CPU\_TYPE CAPACITY8 BCC2 CR. Reply 2: ACK DEV 0 ASCII\_PROGRAM\_DATA BCC2 CR.

## 4.23 Legacy read user program in binary format: Rp

|                       |                                                                  |
|-----------------------|------------------------------------------------------------------|
| <b>Command ID</b>     | PROG-RP-BINARY                                                   |
| <b>Confidence</b>     | A - documented legacy structure                                  |
| <b>Evidence</b>       | S1                                                               |
| <b>Project status</b> | Documented; current FC6A upload data phase not capture-validated |

### Wire structure.

Request phase 1:

```
ENQ DEV 1 R p CPU_TYPE CAPACITY8 BCC2 CR
```

Request phase 2:

```
ENQ DEV 0 R BCC2 CR
```

**Description.** Requests a raw binary user-program upload using the legacy two-phase exchange. The second reply contains binary bytes and therefore must be received with a binary-safe frame reader.

### Fields and interpretation.

|                  |                                             |
|------------------|---------------------------------------------|
| <b>Data type</b> | Lowercase p.                                |
| <b>Reply 2</b>   | Raw program bytes, followed by BCC2 and CR. |

**Reply.** Reply 1 uses ACK result 1 with CPU type and capacity; reply 2 uses ACK result 0 with binary program data.

## 4.24 Open/list an SD-card directory: FR

|                       |                                            |
|-----------------------|--------------------------------------------|
| <b>Command ID</b>     | SD-FR-OPEN                                 |
| <b>Confidence</b>     | A - captured and reproduced in MiSmSDCard  |
| <b>Evidence</b>       | S5                                         |
| <b>Project status</b> | Implemented and used for directory listing |

### Wire structure.

```
ENQ DEV 1 F R PATH_LEN3 PATH_ASCII NUL BCC2 CR
```

**Description.** Opens an SD-card directory and requests its entry count. The path is normalized with a leading slash and terminated by a NUL byte.

### Fields and interpretation.

|                   |                                                                          |
|-------------------|--------------------------------------------------------------------------|
| <b>PATH_LEN3</b>  | Length of PATH_ASCII plus the NUL byte, as three hexadecimal characters. |
| <b>Reply data</b> | Directory-entry count as hexadecimal text.                               |

**Reply.** ACK DEV 0 COUNT\_HEX BCC2 CR. Project code maps NG 22 to path-not-found/not-a-directory.



## 4.25 Read SD-card directory entries: FR20 / FR21

|                       |                                           |
|-----------------------|-------------------------------------------|
| <b>Command ID</b>     | SD-FR-ENTRY                               |
| <b>Confidence</b>     | A - captured and reproduced in MiSmSDCard |
| <b>Evidence</b>       | S5                                        |
| <b>Project status</b> | Implemented                               |

### Wire structure.

Non-final entry request:

ENQ DEV 1 F R 20 CR

Final entry / close request:

ENQ DEV 0 F R 21 CR

**Description.** Reads one directory entry after FR open; continuation 1/20 is used for intermediate entries and continuation 0/21 for the final entry and close. The captured entry requests are a known exception that may be sent without a BCC field.

### Fields and interpretation.

**Entry payload** KIND1 SIZE8 DATE8 TIME6 NAME\_LEN3 NAME, followed by normal reply framing.  
**KIND1** 1 for a directory and 0 for a file in the current parser.

**Reply.** ACK DEV RESULT ENTRY\_DATA BCC2 CR; the result/continuation follows the entry position.

**Notes.** The no-BCC request form is specific to the captured directory-entry sequence and should not be generalized to other commands.

## 4.26 Resolve/check an SD-card path: FR005

|                       |                              |
|-----------------------|------------------------------|
| <b>Command ID</b>     | SD-FR-RESOLVE                |
| <b>Confidence</b>     | A - captured and implemented |
| <b>Evidence</b>       | S5                           |
| <b>Project status</b> | Used by delete/path checks   |

### Wire structure.

ENQ DEV 1 F R 005 PATH\_ASCII BCC2 CR

**Description.** Resolves an SD-card file or directory path before another operation. The captured helper uses the fixed text 005 before the normalized path rather than the length-plus-NUL form used by directory open.

### Fields and interpretation.

**PATH\_ASCII** Normalized path beginning with slash.

**Reply.** Normal ACK/NG framing; returned metadata is operation-dependent.

## 4.27 Delete an SD-card file or directory: FC005

|                       |                              |
|-----------------------|------------------------------|
| <b>Command ID</b>     | SD-FC-DELETE                 |
| <b>Confidence</b>     | A - captured and implemented |
| <b>Evidence</b>       | S5                           |
| <b>Project status</b> | Implemented by deleteSD      |

### Wire structure.

```
ENQ DEV 0 F C 005 PATH_ASCII BCC2 CR
```

**Description.** Deletes the selected SD-card path. The current implementation normally resolves the path first and then sends this final FC request.

### Fields and interpretation.

**PATH\_ASCII** Normalized absolute path on the PLC SD card.

**Reply.** ACK DEV 0 BCC2 CR on success; PLC-side path or state errors use ACK-NG.

## 4.28 Open an SD-card file for reading: Fu

|                       |                                           |
|-----------------------|-------------------------------------------|
| <b>Command ID</b>     | SD-FU-OPEN                                |
| <b>Confidence</b>     | A - captured and reproduced in MiSmSDCard |
| <b>Evidence</b>       | S5                                        |
| <b>Project status</b> | Implemented by readSD/saveSD              |

### Wire structure.

```
ENQ DEV 1 F u PATH_LEN3 PATH_ASCII NUL BLOCK_SIZE3 BCC2 CR
```

**Description.** Opens a file for streamed reading and declares the requested block size. The reply data is decoded as the total file size in hexadecimal.

### Fields and interpretation.

**BLOCK\_SIZE3** Requested bytes per block, 001 through FFF in the implementation.  
**PATH\_LEN3** Path plus NUL length.

**Reply.** ACK DEV 0 FILE\_SIZE\_HEX BCC2 CR.

## 4.29 Read SD-card file blocks: FU

|                       |                                            |
|-----------------------|--------------------------------------------|
| <b>Command ID</b>     | SD-FU-BLOCK                                |
| <b>Confidence</b>     | A - captured and reproduced in MiSmSDCard  |
| <b>Evidence</b>       | S5                                         |
| <b>Project status</b> | Implemented with binary-safe receive logic |

**Wire structure.**

Intermediate block request:  
 ENQ DEV 1 F U BCC2 CR

Final block request:  
 ENQ DEV 0 F U BCC2 CR

**Description.** Requests the next block from an already-open file. The reply starts with a three-character hexadecimal byte count followed by exactly that many raw binary bytes.

**Fields and interpretation.**

**Reply data** COUNT3 BINARY\_DATA.  
**Termination** Use continuation 0 for the final remaining block.

**Reply.** ACK DEV RESULT COUNT3 BINARY\_DATA BCC2 CR.

**4.30 Create/open an SD-card file for writing: FD**

**Command ID** SD-FD-CREATE  
**Confidence** A - captured and implemented  
**Evidence** S5  
**Project status** Implemented in current MiSmSDCard write path

**Wire structure.**

ENQ DEV 1 F D TOTAL\_SIZE8 PATH\_LEN3 PATH\_ASCII NUL BCC2 CR

**Description.** Creates the destination file and declares its complete size before block transfer begins. The path is NUL terminated and its encoded length is carried separately.

**Fields and interpretation.**

**TOTAL\_SIZE8** Unsigned file size as eight hexadecimal characters.  
**PATH\_LEN3** Path plus NUL length.

**Reply.** ACK DEV 0 BCC2 CR on successful creation.

**4.31 Write SD-card file blocks: FD**

**Command ID** SD-FD-BLOCK  
**Confidence** A - captured and implemented  
**Evidence** S5  
**Project status** Implemented with offset validation and final-block handling

**Wire structure.**

Intermediate block:  
 ENQ DEV 1 F D OFFSET8 LENGTH4 BINARY\_DATA BCC2 CR

Final block:  
 ENQ DEV 0 F D OFFSET8 LENGTH4 BINARY\_DATA BCC2 CR

Empty-file finalizer:

ENQ DEV 0 F D 00000000 0000 BCC2 CR

**Description.** Writes binary data at an explicit file offset. Continuation 1 marks an intermediate block, while 0 marks the final block and completes the file.

#### Fields and interpretation.

**OFFSET8** Zero-based byte offset.  
**LENGTH4** Raw bytes carried in BINARY\_DATA.  
**BCC** Calculated across the control/header and binary payload.

**Reply.** Intermediate writes are expected to ACK with result 1; the final write is expected to ACK with result 0.

### 4.32 Create an SD-card directory: FM

**Command ID** SD-FM-MKDIR  
**Confidence** A - captured and implemented  
**Evidence** S5  
**Project status** Implemented by mkdirSD/makedirsSD

#### Wire structure.

ENQ DEV 0 F M PATH\_LEN3 PATH\_ASCII NUL BCC2 CR

**Description.** Creates one directory on the PLC SD card. Recursive creation is implemented by issuing this command once for each missing path component.

#### Fields and interpretation.

**PATH\_LEN3** Path plus NUL length.

**Reply.** ACK DEV 0 BCC2 CR on success.

## 5 Confidence B: strong capture-derived structures

### 5.1 Read extended register area: RA

**Command ID** HMI-RA  
**Confidence** B - repeatedly observed; field grammar strongly matches N-byte reads  
**Evidence** S6  
**Project status** Observed from native IDEC HMI requests; public MiSm extended API still under development

#### Wire structure.

ENQ DEV 0 R A ADDR4\_HEX LEN2 BCC2 CR

Observed normalized examples:

RA A2898, 2 bytes

RA A2904, 2 bytes

**Description.** Reads the FC6A extended register area addressed in hexadecimal rather than the normal decimal D-register notation. It follows the normal N-byte read shape, but TYPE A and the address mapping are not described by the legacy FC4A manual.

#### Fields and interpretation.

**ADDR4\_HEX** Four hexadecimal address digits as observed by the HMI logger.  
**LEN2** Byte count.

**Reply.** Normal ACK data reply with two hexadecimal characters per byte.

**Notes.** The document records the wire structure and observed address form without assigning a D-register equivalence that has not been proven.

## 5.2 Write extended register area: WA

**Command ID** HMI-WA  
**Confidence** B - observed command family; consistent with RA and N-byte writes  
**Evidence** S6  
**Project status** Observed/parsed; complete public implementation pending

#### Wire structure.

```
ENQ DEV 0 W A ADDR4_HEX LEN2 DATA_HEX BCC2 CR
```

**Description.** Writes bytes to the same hexadecimal extended-address space used by RA. Its field ordering follows WD/WM, but TYPE A selects the extended area.

#### Fields and interpretation.

**ADDR4\_HEX** Extended address.  
**DATA\_HEX** 2 x LEN hexadecimal characters.

**Reply.** Normal ACK or ACK-NG framing.

## 5.3 Enable or suspend Force I/O mode: WO

**Command ID** FORCE-WO  
**Confidence** B - capture-derived and implemented  
**Evidence** S5  
**Project status** Implemented as force\_io()

#### Wire structure.

```
ENQ DEV 0 W 0 VALUE BCC2 CR
```

Bodies:

DEVOW01 enable Force I/O  
DEVOW00 disable/suspend Force I/O

**Description.** Controls the PLC Force I/O mode used before individual forced-output commands. The single Boolean field is capture-derived rather than present in the legacy command manual.

**Fields and interpretation.**

**VALUE** ASCII 1 to enable; 0 to disable/suspend.

**Reply.** Normal ACK/NG framing.

**5.4 Force one physical output: W]**

**Command ID** FORCE-WBRACKET  
**Confidence** B - capture-derived and implemented  
**Evidence** S5  
**Project status** Implemented for Q0 through Q7 in current library

**Wire structure.**

```
ENQ DEV 0 W ] BIT4 VALUE BCC2 CR
```

**Description.** Forces one physical output bit after Force I/O mode is enabled. The right-bracket character is the data type/opcode suffix and must be preserved literally.

**Fields and interpretation.**

**BIT4** Four decimal digits identifying the output bit in the current implementation.  
**VALUE** ASCII 0 or 1.

**Reply.** Normal ACK/NG framing.

**5.5 FC6A program segment declaration: WPn**

**Command ID** PROG-WPN  
**Confidence** B - exact capture structure; program download sequence reproduced in project work  
**Evidence** S3, S4  
**Project status** Primary user-program segment declaration known

**Wire structure.**

```
ENQ DEV 1 W P n BASE6 LENGTH4 BCC2 CR
```

Captured body:  
FF1WPn5000009C4C

**Description.** Declares the base and byte length of the primary FC6A user-program segment before its ASCII-hex data blocks. This is a distinct FC6A transfer dialect and should not be collapsed into the legacy WP command.

**Fields and interpretation.**

**BASE6** Six hexadecimal characters; 500000 in the supplied captures.  
**LENGTH4** Segment length as four hexadecimal characters.

**Reply.** Normal ACK framing; data blocks follow after a successful declaration.

## 5.6 FC6A user-program data blocks

|                       |                                                         |
|-----------------------|---------------------------------------------------------|
| <b>Command ID</b>     | PROG-DATA                                               |
| <b>Confidence</b>     | B - exact capture structure                             |
| <b>Evidence</b>       | S3, S4                                                  |
| <b>Project status</b> | Primary and secondary segment data boundaries recovered |

### Wire structure.

Intermediate block:

```
ENQ DEV 1 ASCII_HEX_PROGRAM_DATA BCC2 CR
```

Final block of a segment:

```
ENQ DEV 0 ASCII_HEX_PROGRAM_DATA BCC2 CR
```

**Description.** Carries program bytes as ASCII hexadecimal text with no explicit W command after the continuation field. Full captured blocks contain 1,024 hexadecimal characters, representing 512 binary bytes.

### Fields and interpretation.

|                     |                                                                         |
|---------------------|-------------------------------------------------------------------------|
| <b>Continuation</b> | 1 for intermediate data, 0 for the final block of the declared segment. |
| <b>Data</b>         | Two hexadecimal characters per program byte.                            |

**Reply.** Each block receives the normal ACK result framing.

**Notes.** Hex data beginning with printable letters can look like a command mnemonic to a naive parser; once a segment is declared, framing and expected byte count must control parsing.

## 5.7 FC6A secondary program segment declaration: W;n

|                       |                                          |
|-----------------------|------------------------------------------|
| <b>Command ID</b>     | PROG-WSEMI-N                             |
| <b>Confidence</b>     | B - exact capture structure              |
| <b>Evidence</b>       | S3, S4                                   |
| <b>Project status</b> | Secondary/tail segment declaration known |

### Wire structure.

```
ENQ DEV 1 W ; n BASE6 LENGTH4 BCC2 CR
```

Captured body:

```
FF1W;n5000003E2C
```

**Description.** Declares a secondary or tail program segment using the literal semicolon and lowercase n. It is followed by the same continuation-plus-ASCII-hex block format used after WPn.

### Fields and interpretation.

|                |                                |
|----------------|--------------------------------|
| <b>BASE6</b>   | Six hexadecimal characters.    |
| <b>LENGTH4</b> | Secondary-segment byte length. |

**Reply.** Normal ACK framing.

## 5.8 FC6A transfer status/finalization poll: Ru

|                       |                                                       |
|-----------------------|-------------------------------------------------------|
| <b>Command ID</b>     | PROG-RU                                               |
| <b>Confidence</b>     | B - repeatedly captured; role established by sequence |
| <b>Evidence</b>       | S3, S4                                                |
| <b>Project status</b> | Used repeatedly between and after program segments    |

### Wire structure.

ENQ DEV 0 R u BCC2 CR

Captured body: FF0Ru

**Description.** Polls or advances the FC6A program-transfer state after a segment. WindLDR repeats Ru until the reply changes to the expected ready/finalized state, and sends it again after later transfer phases.

### Fields and interpretation.

**Request payload** None after lowercase u.

**Reply.** ACK DEV 0 STATE\_DATA BCC2 CR. Several state-data patterns were observed, but their individual bit meanings are not assigned in this reference.

## 5.9 Firmware binary stream declaration: STX/WPB

|                       |                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------|
| <b>Command ID</b>     | FW-WPB-DECLARE                                                                          |
| <b>Confidence</b>     | B - exact high-volume capture structure                                                 |
| <b>Evidence</b>       | S4                                                                                      |
| <b>Project status</b> | Two complete stream lengths and block sequences recovered; not reproduced against a PLC |

### Wire structure.

STX DEV 0 W P B BASE8 TOTAL8 BCC2 CR

Captured complete frame:

02 FF0WPB000000000000000109400 7B 0D

**Description.** Declares a firmware binary stream using STX (02h) instead of the normal ENQ control byte. The supplied firmware capture contains declarations for two large streams followed by thousands of WPB blocks.

### Fields and interpretation.

|                     |                                                             |
|---------------------|-------------------------------------------------------------|
| <b>BASE8</b>        | Eight hexadecimal characters; zero in the supplied capture. |
| <b>TOTAL8</b>       | Complete stream size as eight hexadecimal characters.       |
| <b>Control byte</b> | STX 02h, not ENQ 05h.                                       |

**Reply.** The capture repeatedly shows ACK 00 1 0 BCC2 CR.



## 5.10 Firmware binary blocks: STX/WPB

|                       |                                                |
|-----------------------|------------------------------------------------|
| <b>Command ID</b>     | FW-WPB-BLOCK                                   |
| <b>Confidence</b>     | B - exact high-volume capture structure        |
| <b>Evidence</b>       | S4                                             |
| <b>Project status</b> | 6,610 WPB frames decoded from supplied capture |

### Wire structure.

Intermediate block:

```
STX DEV 1 W P B OFFSET8 LENGTH4 BINARY_DATA BCC2 CR
```

Final block:

```
STX DEV 0 W P B OFFSET8 LENGTH4 BINARY_DATA BCC2 CR
```

**Description.** Transfers raw firmware bytes at explicit offsets. Captured full blocks use LENGTH4 = 0100, or 256 bytes, and the final block changes continuation from 1 to 0.

### Fields and interpretation.

|                    |                            |
|--------------------|----------------------------|
| <b>OFFSET8</b>     | Zero-based stream offset.  |
| <b>LENGTH4</b>     | Raw binary payload length. |
| <b>BINARY_DATA</b> | Unencoded bytes.           |

**Reply.** The capture shows ACK 00 1 0 BCC2 CR for the block sequence.

## 6 Confidence C: exact structures with incomplete semantics

### 6.1 HMI multi-bit read: Rl

|                       |                                                                         |
|-----------------------|-------------------------------------------------------------------------|
| <b>Command ID</b>     | HMI-RL                                                                  |
| <b>Confidence</b>     | C - repeatedly observed; exact operand model not documented             |
| <b>Evidence</b>       | S6                                                                      |
| <b>Project status</b> | Parsed by HMI logger; not required by the initial bridge implementation |

### Wire structure.

```
ENQ DEV 0 R 1 ADDR4 LEN2 BCC2 CR
```

Observed normalized example:

```
Rl 12774, 2 bytes
```

**Description.** Reads a packed collection of bits using lowercase l and the N-byte-shaped address/length payload. The project evidence supports a multi-bit result, but the named operand space and its relationship to standard M/R areas remain undocumented.

### Fields and interpretation.

|              |                                   |
|--------------|-----------------------------------|
| <b>ADDR4</b> | Observed four-digit address.      |
| <b>LEN2</b>  | Number of packed bytes requested. |

**Reply.** Normal ACK data reply containing a packed bit field.

## 6.2 FC6A enhanced-security response: continuation-2 WV

|                       |                                                                               |
|-----------------------|-------------------------------------------------------------------------------|
| <b>Command ID</b>     | PROTECT-WV-FC6A                                                               |
| <b>Confidence</b>     | C - exact request length and sequence captured; response algorithm unresolved |
| <b>Evidence</b>       | Project password-oracle captures and tests                                    |
| <b>Project status</b> | Challenge/response framing known; transform deliberately not claimed          |

### Wire structure.

```
ENQ DEV 2 W V RESPONSE20 OPTION BCC2 CR
```

Observed session preamble:

```
DEVORS
```

```
DEVORN
```

```
DEV2WV<20 hexadecimal characters>0
```

**Description.** Carries a 10-byte password-derived response encoded as 20 hexadecimal characters, followed by an option byte. The response is deterministic for a repeated PLC challenge, but it is not plaintext and the transformation is not defined in this reference.

### Fields and interpretation.

|                     |                                                            |
|---------------------|------------------------------------------------------------|
| <b>Continuation</b> | Literal ASCII 2 in the captured enhanced-security request. |
| <b>RESPONSE20</b>   | Twenty hexadecimal characters.                             |
| <b>OPTION</b>       | Observed as ASCII 0 in current failed/validation sessions. |

**Reply.** Incorrect responses produced ACK-NG code 05.

**Notes.** This structure supersedes the legacy eight-byte plaintext WV form on the tested enhanced-security FC6A workflow.

## 6.3 FC6A region/session control: Ri

|                       |                                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------------------|
| <b>Command ID</b>     | PROG-RI                                                                                                 |
| <b>Confidence</b>     | C - exact frame shape and several parameter sets captured; universal semantics incomplete               |
| <b>Evidence</b>       | S3, S4                                                                                                  |
| <b>Project status</b> | Used before program and firmware regions; one parameter set also appears in reset/reinitialization work |

### Wire structure.

```
ENQ DEV 0 R i SELECTOR4 ADDRESS4 LENGTH4 BCC2 CR
```

Captured bodies include:

```
FF0Ri00001FB80004
```

```
FF0Ri000021150082
```

```
FF0Ri00002156007C
```

```
FF0Ri000021940004
```

```
FF0Ri00001FBA000C
```

**Description.** Performs a region- or session-oriented operation before program or firmware transfer stages. The 12-character payload separates cleanly into three four-hex fields, but the exact meaning of each field and whether the operation is read, erase, initialize, or state preparation depends on context and is not generalized here.

**Fields and interpretation.**

**SELECTOR4**      Observed as 0000 in supplied captures.  
**ADDRESS4**        Region address/identifier.  
**LENGTH4**        Region size or operation length.

**Reply.**    ACK DEV 0 REGION\_DATA BCC2 CR; multiple data patterns were observed.

**6.4 Extended SD status poll: RA1F5F0220**

**Command ID**      SD-STATUS-RA  
**Confidence**      C - exact fixed request captured/implemented; field interpretation incomplete  
**Evidence**         S5  
**Project status**    Optional checkSD extended poll

**Wire structure.**

ENQ DEV 0 R A 1F5F 02 20 BCC2 CR

**Description.**    Queries an extended SD-card status location using one fixed request body. The current implementation records the raw reply but does not assign a complete meaning to the trailing 20 field or returned data.

**Fields and interpretation.**

**Fixed payload**    1F5F0220 as currently observed and used.

**Reply.**    Normal ACK data framing; raw data is retained for analysis.

**7 Confidence D: exact observed frames only****7.1 Firmware transfer mode/control: WS77FF05 / WS77FF01**

**Command ID**      FW-WS  
**Confidence**      D - exact frames captured; operation names and field meanings unresolved  
**Evidence**         S4  
**Project status**    Observed only in the with-firmware capture

**Wire structure.**

ENQ DEV 1 W S 77 FF 05 BCC2 CR  
 ENQ DEV 1 W S 77 FF 01 BCC2 CR

**Description.**    Two WS control frames bracket firmware-related phases in the supplied capture. Their exact mode, target, and flag semantics are not assigned because no independent documentation or controlled reproduction is available.

**Fields and interpretation.**

**Fixed observed bytes**  
                       77 FF followed by 05 or 01.

**Reply.** Normal ACK framing was observed.

## 8 Observed HMI request spellings

The HMI logger presents the combined command and type characters as a short mnemonic. The table below keeps that useful spelling while also showing how it maps into the base grammar.

| Mnemonic | Meaning used by logger       | Body template                  | Status                                |
|----------|------------------------------|--------------------------------|---------------------------------------|
| RD       | Read D data registers        | DEVORD ADDR4 LEN2              | Standard N-byte read; high confidence |
| RM       | Read M relay byte block      | DEVORM ADDR4 LEN2              | Standard N-byte packed-bit read       |
| RA       | Read extended register area  | DEVORA ADDR4_HEX LEN2          | Observed FC6A extension               |
| R_       | Read timer records           | DEVOR_ FIRST4 COUNT2           | Official timer-information command    |
| Rl       | Read packed bit collection   | DEVORl ADDR4 LEN2              | Observed; operand mapping incomplete  |
| WD       | Write D data registers       | DEVOWD ADDR4 LEN2 DATA_HEX     | Standard N-byte write                 |
| WM       | Write M relay byte block     | DEVOWM ADDR4 LEN2 DATA_HEX     | Standard packed-bit write             |
| Wm       | Write one M relay bit        | DEVOWm ADDR4 VALUE             | Standard one-bit write                |
| WA       | Write extended register area | DEVOWA ADDR4_HEX LEN2 DATA_HEX | Observed FC6A extension               |

## 9 Error and result codes

### 9.1 Documented ACK-NG codes

| Code | Name                                      | Meaning                                                                                        |
|------|-------------------------------------------|------------------------------------------------------------------------------------------------|
| 01   | Program size/capacity error               | Declared program capacity is not acceptable.                                                   |
| 02   | Protection error                          | The requested program read/write is blocked by configured protection.                          |
| 03   | RUN error                                 | A program write was attempted while the PLC was running.                                       |
| 04   | CRC error                                 | Program CRC validation failed.                                                                 |
| 05   | Protect-code error                        | The protection response/code did not match, or an invalid protection transition was requested. |
| 06   | Data-range error                          | Address, quantity, or range is invalid.                                                        |
| 07   | Timer/counter preset change error         | A write attempted to change a preset that is designated through a data register.               |
| 08   | Calendar/clock data error                 | A clock field value is invalid.                                                                |
| 09   | Data-clear error                          | The selected area cannot be cleared in the requested state.                                    |
| 10   | Data error                                | Payload data is outside the command's accepted encoding or values.                             |
| 11   | Setting error                             | A user-communication setting is invalid.                                                       |
| 12   | CPU-type error                            | The declared CPU type does not match the connected PLC.                                        |
| 22   | SD path/state error (project observation) | Current SD helper maps this code to a missing path or non-directory during directory open.     |

### 9.2 Documented NAK communication errors

| Code | Name                          | Meaning                                                                                                                                                                    |
|------|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 00   | BCC error                     | Received checksum does not match the calculated checksum.                                                                                                                  |
| 01   | Frame error                   | Character/frame length or stop-bit framing is invalid.                                                                                                                     |
| 02   | Send/receive error            | Parity or overrun error.                                                                                                                                                   |
| 03   | Command error                 | Unsupported command or type.                                                                                                                                               |
| 04   | Procedure/data-quantity error | Request order or data quantity does not match the expected transaction.                                                                                                    |
| 10   | FC6A project handling         | Current MiSm transport code treats NAK 10 as a signal to retry the alternate request-BCC calculation mode; an official meaning has not been established in the source set. |

## 10 Coverage matrix

| Command ID        | Grade | Evidence           | Current project coverage                                                                  |
|-------------------|-------|--------------------|-------------------------------------------------------------------------------------------|
| CORE-RNBYTE       | A     | S1, S5, S6         | Implemented in MiSmSerial/MiSmTCP; HMI-originated RD and RM observed                      |
| CORE-WNBYTE       | A     | S1, S5, S6         | Implemented; HMI-originated WD, WM, and calendar WW observed                              |
| CORE-R1BIT        | A     | S1, S3, S4, S5     | Implemented; Rm observed in program-transfer captures                                     |
| CORE-W1BIT        | A     | S1, S3, S4, S5, S6 | Implemented; used for HMI buttons and PLC stop/run control                                |
| SYS-RS            | A     | S1, S3, S4, S5     | Implemented and used as a session preflight                                               |
| SYS-RK            | A     | S1, S5             | Implemented/verified; observed reply included current and maximum values                  |
| SYS-RN            | A     | S1, S3, S4, S5     | Implemented and used before protection/program operations                                 |
| SYS-RE            | A     | S1                 | Structure covered; not a focus of current capture work                                    |
| SYS-CLEAR         | A     | S1                 | Structure covered; destructive forms intentionally separated from routine read/write APIs |
| CLOCK-RW          | A     | S1                 | Structure covered; corresponding WW was captured                                          |
| CLOCK-WW          | A     | S1, S3, S4         | Observed during WindLDR sessions                                                          |
| TC-RINFO          | A     | S1, S6             | Observed for HMI timer reads                                                              |
| TC-RCINFO         | A     | S1                 | Structure covered; no current HMI example retained                                        |
| TC-RA             | A     | S1                 | Structure covered                                                                         |
| TC-RB             | A     | S1                 | Structure covered                                                                         |
| TC-RD             | A     | S1                 | Structure covered                                                                         |
| TC-RE             | A     | S1                 | Structure covered                                                                         |
| TC-WQ             | A     | S1                 | Structure covered                                                                         |
| PROTECT-WV-LEGACY | A     | S1                 | Documented; distinct from the FC6A enhanced-security WV variant                           |
| PROG-WP-ASCII     | A     | S1                 | Documented; not the FC6A WPn transfer used in current captures                            |
| PROG-WP-BINARY    | A     | S1                 | Documented; not the FC6A WPn transfer used in current captures                            |
| PROG-RP-ASCII     | A     | S1                 | Documented; current FC6A upload data phase not capture-validated                          |
| PROG-RP-BINARY    | A     | S1                 | Documented; current FC6A upload data phase not capture-validated                          |
| SD-FR-OPEN        | A     | S5                 | Implemented and used for directory listing                                                |
| SD-FR-ENTRY       | A     | S5                 | Implemented                                                                               |
| SD-FR-RESOLVE     | A     | S5                 | Used by delete/path checks                                                                |
| SD-FC-DELETE      | A     | S5                 | Implemented by deleteSD                                                                   |

|                 |   |                                            |                                                                                                         |
|-----------------|---|--------------------------------------------|---------------------------------------------------------------------------------------------------------|
| SD-FU-OPEN      | A | S5                                         | Implemented by readSD/saveSD                                                                            |
| SD-FU-BLOCK     | A | S5                                         | Implemented with binary-safe receive logic                                                              |
| SD-FD-CREATE    | A | S5                                         | Implemented in current MiSmSDCard write path                                                            |
| SD-FD-BLOCK     | A | S5                                         | Implemented with offset validation and final-block handling                                             |
| SD-FM-MKDIR     | A | S5                                         | Implemented by mkdirSD/makedirsSD                                                                       |
| HMI-RA          | B | S6                                         | Observed from native IDEC HMI requests; public MiSm extended API still under development                |
| HMI-WA          | B | S6                                         | Observed/parsed; complete public implementation pending                                                 |
| FORCE-WO        | B | S5                                         | Implemented as force_io()                                                                               |
| FORCE-WBRACKET  | B | S5                                         | Implemented for Q0 through Q7 in current library                                                        |
| PROG-WPN        | B | S3, S4                                     | Primary user-program segment declaration known                                                          |
| PROG-DATA       | B | S3, S4                                     | Primary and secondary segment data boundaries recovered                                                 |
| PROG-WSEMI-N    | B | S3, S4                                     | Secondary/tail segment declaration known                                                                |
| PROG-RU         | B | S3, S4                                     | Used repeatedly between and after program segments                                                      |
| FW-WPB-DECLARE  | B | S4                                         | Two complete stream lengths and block sequences recovered; not reproduced against a PLC                 |
| FW-WPB-BLOCK    | B | S4                                         | 6,610 WPB frames decoded from supplied capture                                                          |
| HMI-RL          | C | S6                                         | Parsed by HMI logger; not required by the initial bridge implementation                                 |
| PROTECT-WV-FC6A | C | Project password-oracle captures and tests | Challenge/response framing known; transform deliberately not claimed                                    |
| PROG-RI         | C | S3, S4                                     | Used before program and firmware regions; one parameter set also appears in reset/reinitialization work |
| SD-STATUS-RA    | C | S5                                         | Optional checkSD extended poll                                                                          |
| FW-WS           | D | S4                                         | Observed only in the with-firmware capture                                                              |

## 11 Known coverage boundaries

- The supplied UploadCapture.pcapng did not yield a usable PLC Maintenance Protocol stream in the current parser. Therefore, the official RP/Rp upload structures are included, but no distinct FC6A upload data-phase extension is claimed from that file.
- The enhanced-security WV frame boundary and response length are known; the challenge-to-response transformation is not included because it has not been proven.
- RA/WA hexadecimal extended addressing and Rl packed-bit behavior are observed, but this reference does not invent a complete address-to-device map.
- Ri parameter boundaries are known, and its position in transfer/reinitialization sequences is known; a single universal operation name is not assigned.
- WS frames and the state payloads returned by Ru are retained as exact observations without unsupported bit-field interpretations.

## A Evidence sources

| ID | Source                                 | Use                                                                                                          |
|----|----------------------------------------|--------------------------------------------------------------------------------------------------------------|
| S1 | fc4a_protocol_im.pdf                   | Official MicroSmart Communication Protocol grammar, command list, request/reply structures, and error codes. |
| S2 | idec-fcsixa-windldr-com-usermanual.pdf | FC6A maintenance communication capabilities and Ethernet server configuration, including port 2101.          |
| S3 | RemoteProgramDownload.pcapng           | FC6A user-program transfer sequence: RS, Rm, RN, WW, Wm, Ri, WPn, data blocks, Ru, and W;n.                  |

|    |                                              |                                                                                                                                 |
|----|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| S4 | RemoteProgramDownload<br>WithFirmware.pcapng | Program-transfer sequence plus WS and STX/WPB firmware declarations and blocks.                                                 |
| S5 | Current project implementations              | MiSmSerial/MiSmTCP framing and standard commands; MiSmS-DCard FR/FC/Fu/FU/FD/FM structures; capture-derived Force I/O commands. |
| S6 | HMI logger observations                      | Native HMI request mnemonics RD, RM, RA, R_, RI, WA, WD, WM, and Wm, with observed addresses and byte counts.                   |

## B Packet-capture facts used in this revision

- The program-only capture contains one WPn declaration, one W;n declaration, repeated Ru polls, and a block stream whose full frames carry 512 binary bytes as 1,024 ASCII hexadecimal characters.
- The with-firmware capture contains 6,610 STX/WPB frames. Full firmware blocks carry 0100h bytes, or 256 raw bytes, and final blocks use continuation 0.
- A verified BCC example is <ENQ>FF0RS34<CR>; XOR of bytes 05h, “FF0RS” is 34h. A firmware example is <STX>FF0WPB00000000000000001094007B<CR>.

## C Compact grammar for automated readers

```

NORMAL_REQUEST  := 0x05 DEV2 CONT1 BODY BCC2 0x0D
FIRMWARE_REQUEST:= 0x02 DEV2 CONT1 "WPB" FW_FIELDS BCC2 0x0D
ACK_REPLY       := 0x06 DEV2 RESULT1 DATA BCC2 0x0D
NAK_REPLY       := 0x15 DEV2 "0" ERROR2 BCC2 0x0D

RESULT1 := "0" | "1" | "2"
CONT1   := "0" | "1" | observed-enhanced-security "2"
BCC2    := ASCII_HEX2(XOR(bytes from control byte through last body/data byte))

READ_N_BYTES    := DEV2 "OR" TYPE1 ADDR4 LEN2
WRITE_N_BYTES   := DEV2 "OW" TYPE1 ADDR4 LEN2 HEXDATA(2*LEN)
READ_ONE_BIT    := DEV2 "OR" LOWER_TYPE1 ADDR4
WRITE_ONE_BIT    := DEV2 "OW" LOWER_TYPE1 ADDR4 BOOL1

```